

MSPR TPRE814 - Bloc 4

FutureKawa

Application IoT de supervision des stocks et des conditions de stockage

Rendu technique et fonctionnel

Équipe projet : Antoine GARNIER, Thibault AUTEXIER, Issam HARNOUFI, Zaid ABABOU, Ali WARI

Version du 30 juin 2026

Dossier rédigé à partir du projet livré, des captures applicatives et des preuves d'exécution Docker/MQTT.

Table des matières

- 1. Contexte, besoin et périmètre du POC
- 2. Architecture distribuée et choix techniques
- 3. Modèle de données, API et intégration multi-pays
- 4. Chaîne IoT MicroPython, MQTT et persistance
- 5. Interface web, alertes et expérience utilisateur
- 6. Tests, CI Jenkins et preuves d'exécution
- 7. Conduite du changement et documentation
- 8. Validation de la grille et points restant à sécuriser
- 9. Annexes de démonstration

1. Contexte et périmètre

FutureKawa doit suivre ses lots de café et contrôler les conditions de stockage dans plusieurs pays. Le risque métier principal est simple : une température ou une humidité hors seuil peut dégrader la qualité du stock avant que les équipes ne s'en rendent compte. Le POC livré répond donc à deux besoins : centraliser les informations de stock et faire remonter automatiquement les mesures IoT.

Le projet a été conçu comme une solution distribuée. Chaque pays dispose de sa propre API et de sa propre base MySQL, tandis qu'une API centrale joue le rôle du siège et expose les données à l'interface React. Le flux IoT suit le cours : un ESP32 équipé d'un capteur DHT11 publie en MQTT vers Mosquitto ; un bridge transforme ensuite le message MQTT en écriture REST dans l'API pays.

Périmètre fonctionnel livré

Fonction	Statut	Preuve
Gestion des exploitations et entrepôts	Livré	Pages Exploitations et Entrepôts + API centrale par pays
Gestion des lots	Livré	Cartes de lots, recherche, historique et statut
Mesures température/humidité	Livré	ESP32 MicroPython, MQTT, simulateur et endpoint POST /mesures
Alertes	Livré	Statut en alerte, affichage UI, notification e-mail en mode log ou SMTP
Architecture multi-pays	Livré	Brazil, Colombia et Ecuador dans Docker Compose
CI/CD	Livré	Jenkinsfile + validation Docker Compose + builds

2. Architecture distribuée

L'architecture a été gardée volontairement lisible pour une soutenance : les responsabilités sont séparées et chaque composant peut être lancé localement avec Docker Compose. Ce choix permet de démontrer le projet sans dépendre d'un cloud externe.

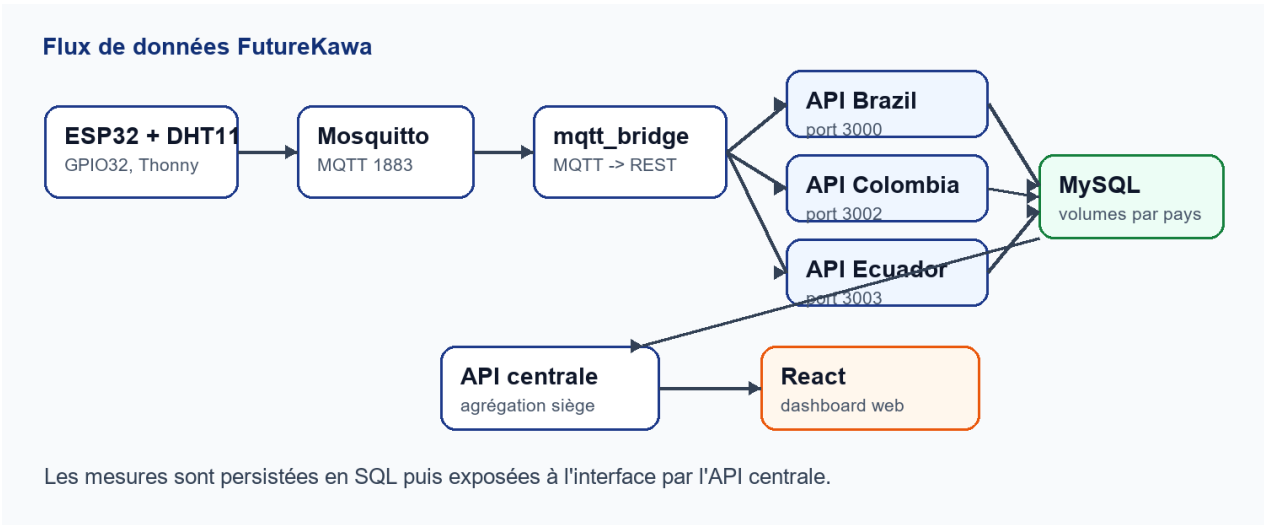


Figure 1 - Architecture technique du flux IoT vers l'interface web.

Services Docker

Service	Rôle
app_central	Interface React servie sur le port 8080.
api_central	API siège sur le port 3001 ; agrège Brazil, Colombia et Ecuador.
api_brazil / api_colombia / api_ecuador	APIs pays isolées, chacune connectée à sa base.
mysql_Brazil / mysql_Colombia / mysql_Ecuador	Persistence SQL par pays avec volumes Docker.
mqtt_broker	Broker Mosquitto local sur le port 1883.
mqtt_bridge	Abonné MQTT qui poste les mesures dans l'API Brazil par défaut.
iot_simulator	Génération de mesures de démonstration quand le capteur n'est pas branché.

Services Docker actifs

NAME	SERVICE	STATUS	PORTS
api_brazil	api_brazil	Up About a minute	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
api_central	api_central	Up About a minute	0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
api_colombia	api_colombia	Up About a minute	0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
api_ecuador	api_ecuador	Up About a minute	0.0.0.0:3003->3003/tcp, [::]:3003->3003/tcp
app_central	app_central	Up About a minute	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
iot_simulator	iot_simulator	Up About a minute	5678/tcp
mqtt_bridge	mqtt_bridge	Up About a minute	
mqtt_broker	mqtt_broker	Up 17 minutes	0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp
mysql_Brazil	db_Brazil	Up 17 minutes (healthy)	0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
mysql_Colombia	db_Colombia	Up 17 minutes (healthy)	0.0.0.0:3308->3306/tcp, [::]:3308->3306/tcp
mysql_Ecuador	db_Ecuador	Up About a minute (healthy)	0.0.0.0:3309->3306/tcp, [::]:3309->3306/tcp

Figure 2 - Services actifs après ajout du troisième pays Ecuador.

3. Données et API

Le modèle de données reste volontairement court : exploitation, entrepôt, lot et mesure. Cela suffit pour relier un stock à son lieu physique et à son historique de température/humidité. Les statuts sont calculés côté API afin de garder une règle unique pour le frontend, les tests et les notifications.

Table	Contenu principal
exploitations	Nom de l'exploitation.
entrepots	Nom et rattachement à une exploitation.
lots	Entrepôt, date de stockage et statut : conforme, en alerte ou périmé.
mesures	Entrepôt, température, humidité, timestamp et statut.

Routes principales

```
GET /:country/exploitations
GET /:country/entrepots
GET /:country/lots
GET /:country/mesures/entrepot/:id/latest
POST /mesures (API pays, réception des mesures IoT)
```

La preuve multi-pays a été renforcée en ajoutant Ecuador dans Docker Compose et dans les mappings de l'API centrale. Le frontend avait déjà la sélection Équateur ; le routage backend manquant a été ajouté pour les exploitations, entrepôts, lots et mesures.

```
API centrale - donnees Equateur

{
  "value": [
    {
      "id_exploitation": "1",
      "nom": "Exploitation Defensive Appetizer"
    },
    {
      "id_exploitation": "2",
      "nom": "Exploitation Untrue Attention"
    },
    {
      "id_exploitation": "3",
      "nom": "Exploitation Minor Hacienda"
    }
  ],
  "Count": 3
}
```

Figure 3 - L'API centrale expose les exploitations Ecuador après seed de démonstration.

4. Chaîne IoT

La partie embarquée est écrite en MicroPython pour correspondre à la demande du cours et à l'utilisation de Thonny. Le capteur DHT11 est branché sur l'ESP32 avec VCC sur 3V3, GND sur GND et DATA sur GPIO32. Le script lit le capteur, construit un payload JSON puis le publie sur le topic MQTT futurekawa/mesures.

Rôle du script MicroPython

- Connexion Wi-Fi avec les identifiants du fichier config.py.
- Connexion au broker Mosquitto sur le port 1883.
- Lecture DHT11 : température et humidité.
- Publication périodique d'un JSON : id_entrepot, temperature, humidite.

```
{"id_entrepot":1,"temperature":26.5,"humidite":55}  
Topic MQTT : futurekawa/mesures  
Fichier : iot/esp32-dht11/micropython/main_mqtt.py
```

À domicile, sans capture Thonny exploitable, le flux matériel est sécurisé par un test MQTT reproductible : mosquitto_pub publie le même message que l'ESP32, puis mqtt_bridge le persiste dans l'API. Cela prouve le maillon MQTT -> API -> MySQL, qui est la partie critique pour le rendu.

```
Preuve MQTT -> API -> MySQL  
  
Connecting MQTT: mqtt_broker 1883  
MQTT connected: Success  
Subscribed to: futurekawa/mesures  
mqtt: futurekawa/mesures {id_entrepot:1,temperature:26.5,humidite:55}  
posted: 201  
{ "id_entrepot":1,"temperature":26.5,"humidite":55,"statut":"conforme","id_mesure":26112,"timestamp":"2026-06-30T21:36:42.000Z" }
```

Figure 4 - Le bridge reçoit une mesure MQTT et obtient une création HTTP 201.

```
Derniere mesure lue via API centrale  
  
{  
  "id_mesure": 26116,  
  "id_entrepot": "1",  
  "temperature": 27.22,  
  "humidite": 54.91,  
  "timestamp": "2026-06-30T21:37:01.000Z",  
  "statut": "conforme"  
}
```

Figure 5 - Dernière mesure relue via l'API centrale après persistance.

5. Interface web

L'interface React sert de poste de supervision. Elle est pensée pour une démonstration rapide : sélection du pays, choix de l'exploitation, lecture d'un entrepôt, consultation des lots, mesures récentes et alertes. Les pages Exploitations et Entrepôts ont été enrichies pour éviter de cacher l'information dans des tableaux bruts.

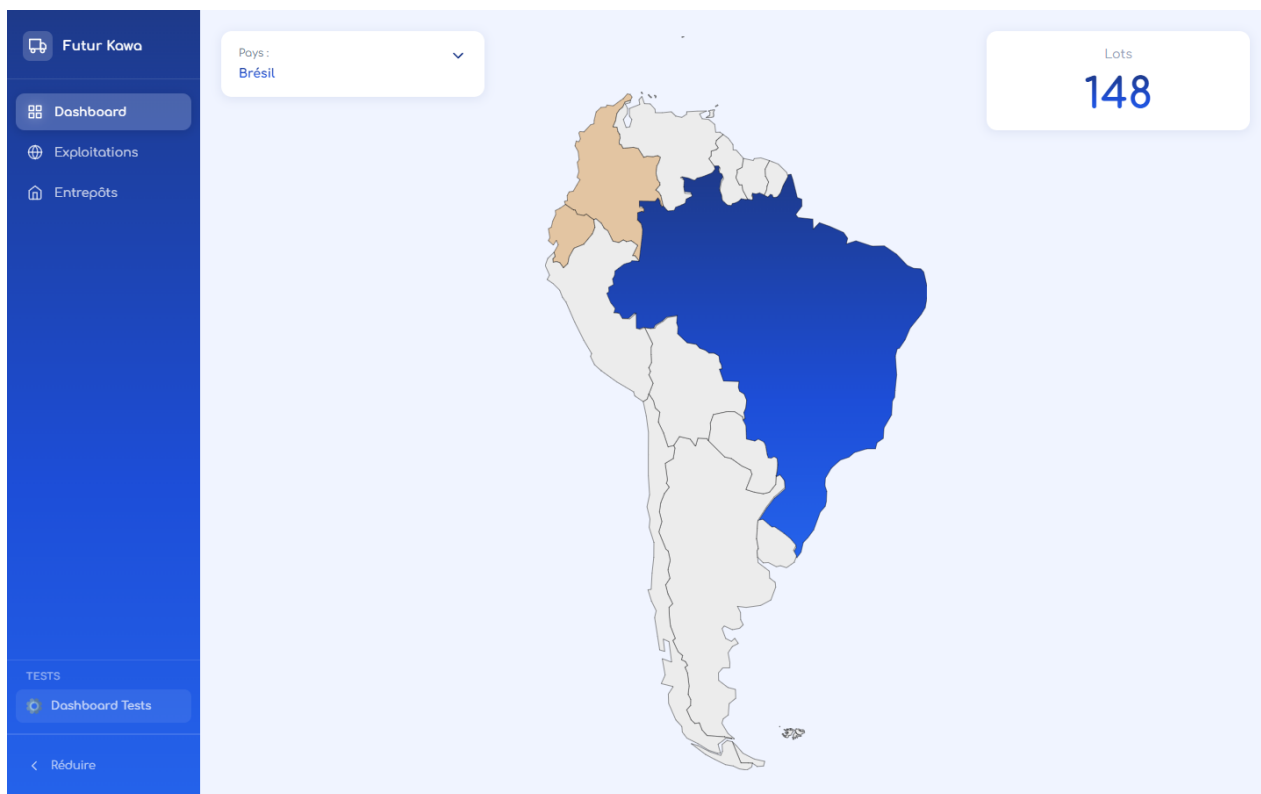


Figure 6 - Dashboard avec sélection du Brésil.

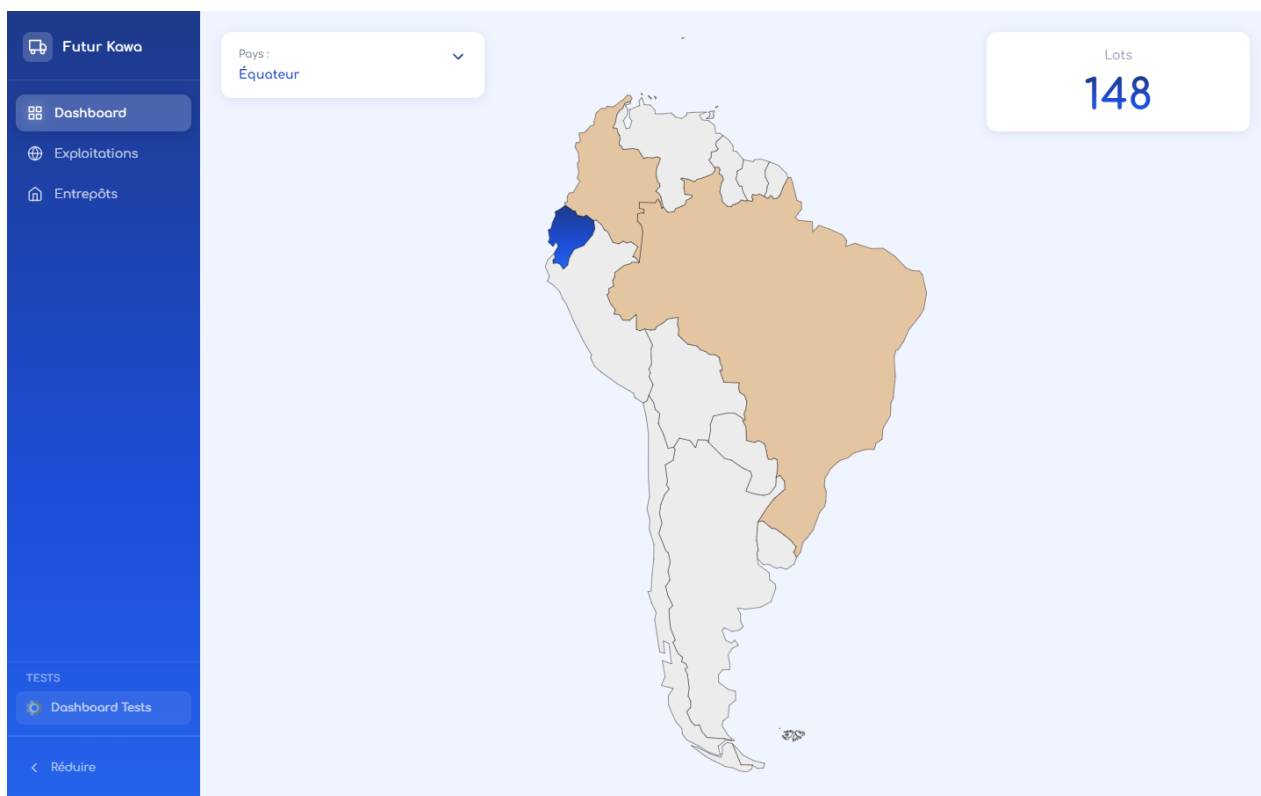


Figure 7 - Dashboard avec sélection de l'Équateur, troisième pays du périmètre.

Page Exploitations

La page Exploitations affiche les indicateurs rapides de l'entrepôt sélectionné. Le graphe a été corrigé pour rester lisible quand l'historique contient beaucoup de points : seules les mesures récentes sont prises, puis regroupées en buckets moyens lorsque le volume devient trop grand. L'utilisateur voit donc une tendance compréhensible au lieu d'un bloc visuel illisible.

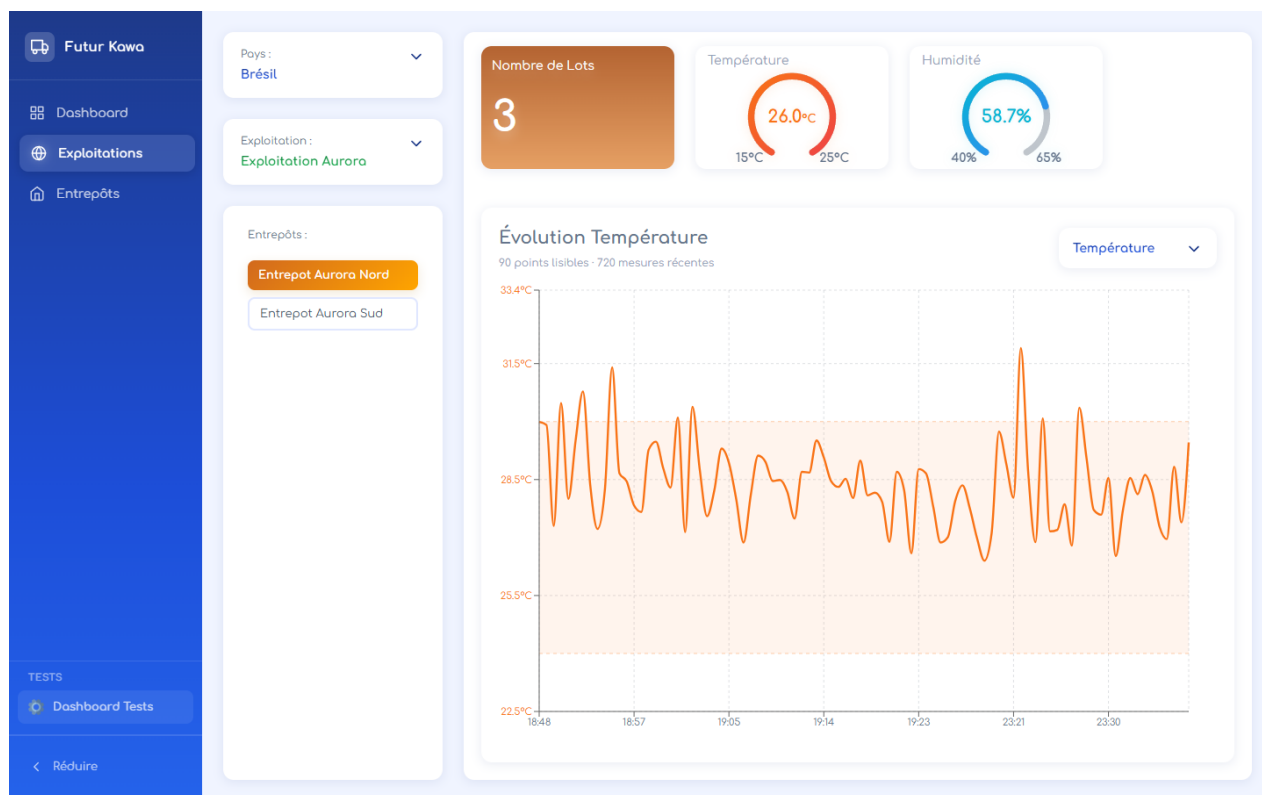


Figure 8 - Page Exploitations : jauges et graphe lisible après regroupement des mesures.

Page Entrepôts

La page Entrepôts permet de cliquer sur chaque entrepôt et de voir le détail : lots, température, humidité, dernière mesure et historique récent. Les alertes visibles dans l'historique donnent le contexte nécessaire : valeur mesurée, statut et moment de la mesure.

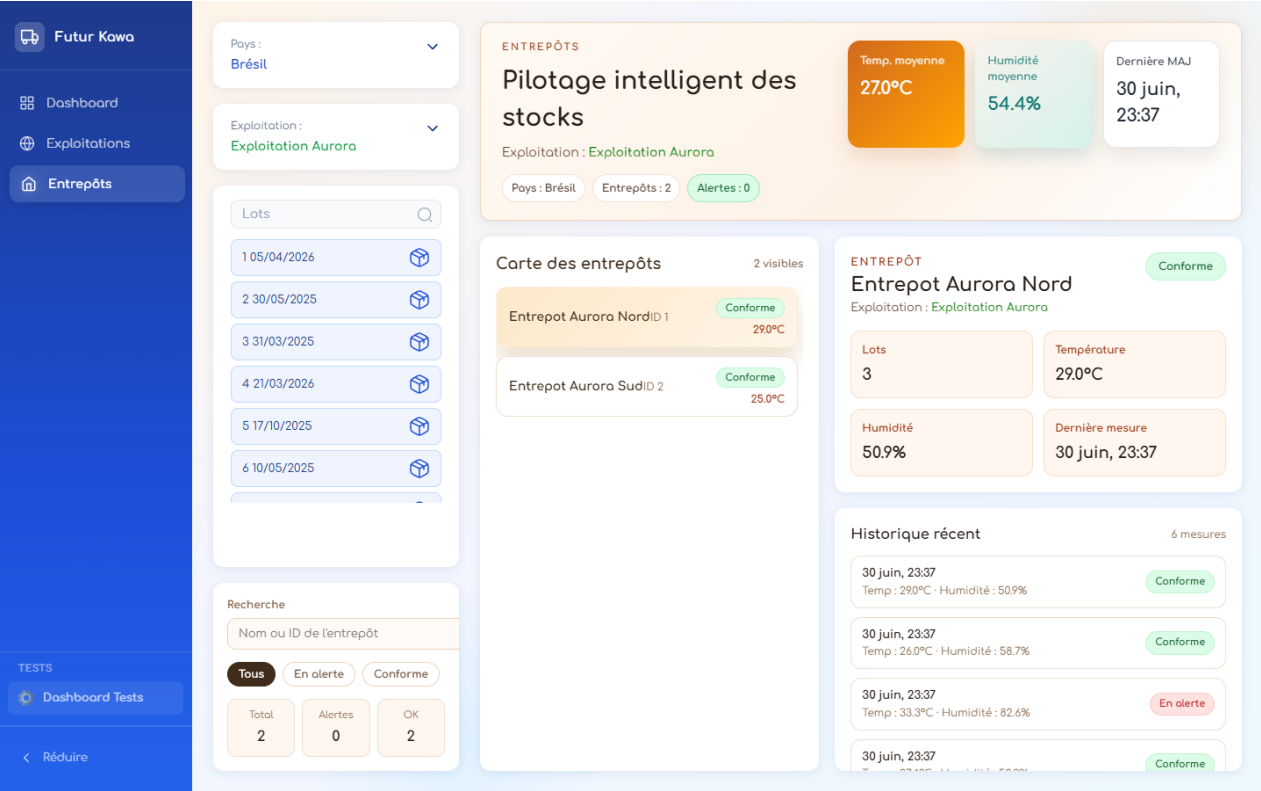


Figure 9 - Page Entrepôts : détail opérationnel et historique des mesures.

6. Alertes

Les règles d'alerte sont centralisées côté API pays. Une mesure devient en alerte si la température sort de l'intervalle 24-30 °C ou si l'humidité sort de l'intervalle 50-60 %. Les lots peuvent aussi être signalés selon leur ancienneté, afin de faire remonter les risques de péremption.

Notification e-mail

Pour éviter de mettre des identifiants SMTP dans le dépôt, le comportement par défaut est le mode log. Il génère exactement le contenu de l'e-mail attendu : sujet, entrepôt, mesure, température, humidité, statut et seuils. En production, le mode SMTP s'active par variables d'environnement : ALERT_EMAIL_MODE=smtp, SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS et ALERT_EMAIL_TO.

```
Preuve alerte e-mail en mode log

EMAIL ALERT LOG
Subject: [FutureKawa] Alerte entrepot 1
Une mesure est en alerte pour l'entrepot 1.
Mesure: 26114
Temperature: 33.56 C
Humidite: 85.36 %
Statut: en alerte
Seuils attendus: temperature 24-30 C, humidite 50-60 %.
```

Figure 10 - Contenu de l'alerte e-mail validé en mode log.

Navigation depuis les alertes

Le choix produit retenu est de ne pas afficher une alerte isolée sans contexte. Une alerte doit ramener l'utilisateur vers la zone métier concernée : pays, exploitation, entrepôt et historique des mesures. C'est plus logique pour un responsable d'exploitation, car il peut immédiatement comparer la dernière mesure avec les lots présents.

7. Tests et CI

Le plan de tests est organisé autour du chemin critique : démarrage Docker, écriture API conforme, écriture API en alerte, publication MQTT, interface web, build local et vérification multi-pays. Les commandes sont reproductibles dans PowerShell et documentées dans docs/tests/plan_de_tests.md.

Test	Résultat observé
Docker Compose	Stack dev reconstruite et active, compose config --quiet sans erreur.
API alerte	POST /mesures avec 34 °C et 84 % retourne statut en alerte.
MQTT	mosquitto_pub -> mqtt_bridge -> POST API -> HTTP 201.
API centrale	Lecture /brazil/... et /ecuador/... fonctionnelle.
Frontend	Build Vite terminé ; screenshots Dashboard, Exploitations et Entrepôts générés.
Jenkins	Jenkinsfile présent : validate compose, build APIs, build frontend, build bridge.

Pipeline Jenkins

Le Jenkinsfile vérifie les éléments qui casseraient une livraison : configuration Docker Compose, build country API, build central API, build frontend et build de l'image mqtt_bridge. Le pipeline ne remplace pas les tests manuels IoT, mais il empêche de pousser une version qui ne compile plus.

```
stages:
- Validate Docker Compose
- Build Country API
- Build Central API
- Build Frontend
- Build MQTT Bridge Image
```

8. Conduite du changement

Le projet ne se limite pas au code : les responsables d'exploitation doivent comprendre ce que signifie une alerte et comment réagir. Le plan de conduite du changement prévoit donc une démonstration courte, une fiche réflexe, des référents pays et une période pilote sur un entrepôt.

Public	Besoin traité
Responsables d'exploitation	Lire les alertes et prioriser les actions.
Équipes entrepôt	Comprendre les mesures et vérifier le capteur.
Direction qualité	Suivre la conformité des conditions de stockage.
DSI	Maintenir Docker, APIs, bases SQL et MQTT.
Siège	Voir une supervision multi-pays centralisée.

Les documents livrés couvrent le cadrage, l'architecture, les tests, Jenkins, les alertes e-mail/log et la conduite du changement. Cela répond à l'attendu de livrables projet, pas seulement à une démonstration technique.

9. Validation de la grille

Attendu de la grille	Statut	Éléments de preuve
Recueil du besoin et cadrage	Validé	Questionnaire phase 2, périmètre du POC, besoins FutureKawa explicités.
Architecture applicative distribuée	Validé	API centrale + APIs pays Brazil/Colombia/Ecuador + bases MySQL séparées + Docker Compose.
Développement web / API / IoT	Validé	React, NestJS, MySQL, ESP32 MicroPython, Mosquitto MQTT, bridge Python.
Intégration logicielle au SI	Validé	API centrale agrégatrice, routes par pays, persistance SQL et interface de supervision.
Tests et plan de tests	Validé	Plan de tests T01 à T08, preuves API/MQTT/UI, compose config et builds.
CI/CD Jenkins	Validé	Jenkinsfile à la racine et documentation docs/ci/jenkins.md.
Documentation utilisateur et technique	Validé	README, dossier technique, guide MQTT/ESP32, alertes, commandes de lancement.
Conduite du changement	Validé	Plan d'information, communication, formation, participation et indicateurs.

Points à sécuriser avant la remise

- Si le matériel est disponible le jour J, refaire une capture Thonny montrant published: {...}.
- Si le jury exige un vrai e-mail reçu, ajouter des identifiants SMTP dans l'environnement local puis relancer le test alerte.
- Éviter de présenter l'ancien dossier non suivi Smart Tracker, qui n'appartient pas au rendu.

10. Annexes

Lancement du projet

```
docker compose --profile dev up --build -d  
http://localhost:8080
```

Seed Ecuador

```
docker compose exec -e SEED_TRUNCATE=true -e SEED_RANDOM_SEED=814 api_ecuador npm run seed
```

Test alerte API

```
Invoke-RestMethod -Method Post -Uri "http://localhost:3000/mesures" -ContentType "application/json"  
-Body '{"id_entrepot":1,"temperature":34,"humidite":84}'
```

Test MQTT sans ESP32

```
docker exec mqtt_bridge mosquitto_pub -h localhost -p 1883 -t futurekawa/mesures -m  
"{id_entrepot:1,temperature:26.5,humidite:55}"  
docker logs --tail 30 mqtt_bridge
```

Fichiers importants

- `iot/esp32-dht11/micropython/main_mqtt.py` : code ESP32 MicroPython.
- `iot/mqtt-bridge/bridge.py` : bridge MQTT vers API REST.
- `country/api/src/alerts/alert-notification.service.ts` : notification alertes.
- `central/app/src/page/Exploitations/StatsCard/StatsCard.tsx` : graphe lisible par regroupement.
- `docker-compose.yml` : orchestration des trois pays, MQTT et application web.
- `Jenkinsfile` : pipeline CI.